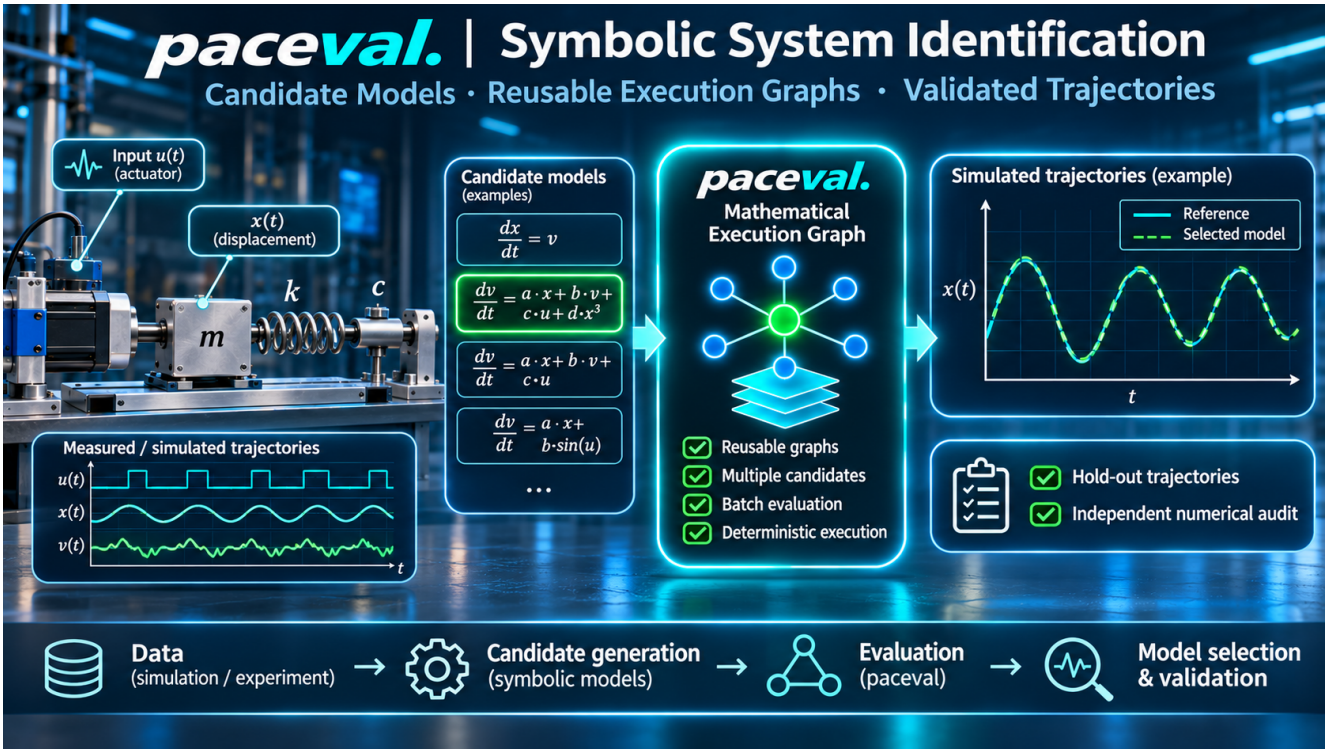
Reusable Mathematical Candidate Evaluation in Symbolic System Identification

A C++ research demonstrator using official paceval execution graphs, nonlinear spring–mass and servo-actuator models, independent trajectory splits and numerical audit.



Conceptual marketing illustration, not a plot of measured experimental hardware.

Executive research result

The implemented program evaluates 3,000 explicit, parameterized mathematical candidates using seven reusable paceval computation graphs and two additional TINC graphs. It selects candidate dynamics from simulated training and validation records, then integrates the selected expressions on four completely held-out trajectories. The ground-truth structures were deliberately included in the candidate pools and were recovered in both cases.

All 7,325 systematic native C++ checks passed; 144 independent 110-digit point evaluations matched within the declared tolerance. All 24 sampled TINC intervals contained their respective high-precision reference values. The release makes no execution-speed comparison or claim of novel physical discovery.

1. Scientific basis and application boundary

Symbolic system identification seeks compact governing equations from state trajectories and known inputs. SINDy (Brunton, Proctor and Kutz, 2016) and its modern implementation guidance distinguish derivative estimation, candidate generation, sparse model selection and independent dynamical validation. This study isolates one reusable mathematical expression execution component within that workflow.

2. Two synthetic nonlinear systems

Both models use a state pair (x, v) with dx/dt = v and an explicitly prescribed input u(t). The spring–mass reference follows a Duffing-like cubic stiffness term, while the illustrative actuator follows cubic velocity damping. Their accelerations are:

Spring–mass: $dv/dt = -1.8\,x - 0.28\,v + 1.10\,u - 0.075\,x^3$

Servo actuator: $dv/dt = -1.45\,x - 0.62\,v + 1.65\,u - 0.16\,v^3$

These coefficients are synthetic dimensionless numbers, not laboratory-identified hardware parameters. All reference trajectories are generated with a native C++ RK4 solver at timestep 0.01; the screening data are recorded every 0.04 time units.

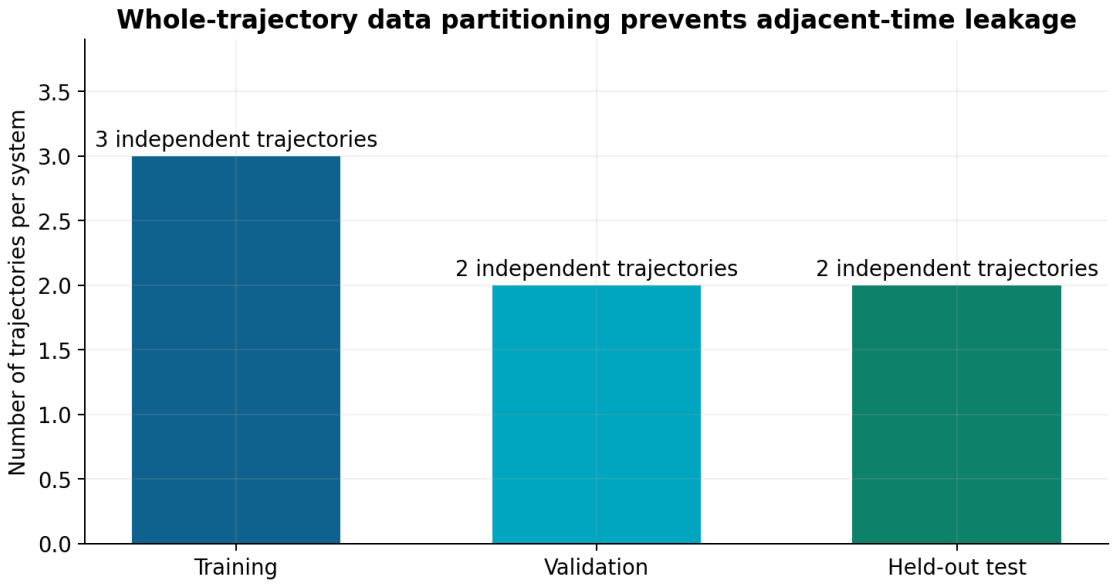


Figure 1. Per-system split: three entire training trajectories, two entire validation trajectories and two entirely independent held-out trajectories.

3. Candidate generation and paceval graph reuse

Seven explicit formula families are examined: linear, cubic-stiffness, quadratic-stiffness, sinusoidal-restoring, cubic-damping, position–velocity cross-coupling and tanh saturation. Candidate instances are defined by their family and four numerical coefficients (a,b,c,d). A fixed seeded generator produces 1,500 instances per system, including exactly one deliberately planted reference candidate for each. No coefficient fitting or evolutionary search is claimed.

The seven expressions share the variable signature x v u a b c d. paceval creates one computation object per expression family. All other candidate variations are processed by repeated batch evaluation with new input and parameter sets. This demonstrates formula reuse without suggesting that paceval itself generates scientific hypotheses.

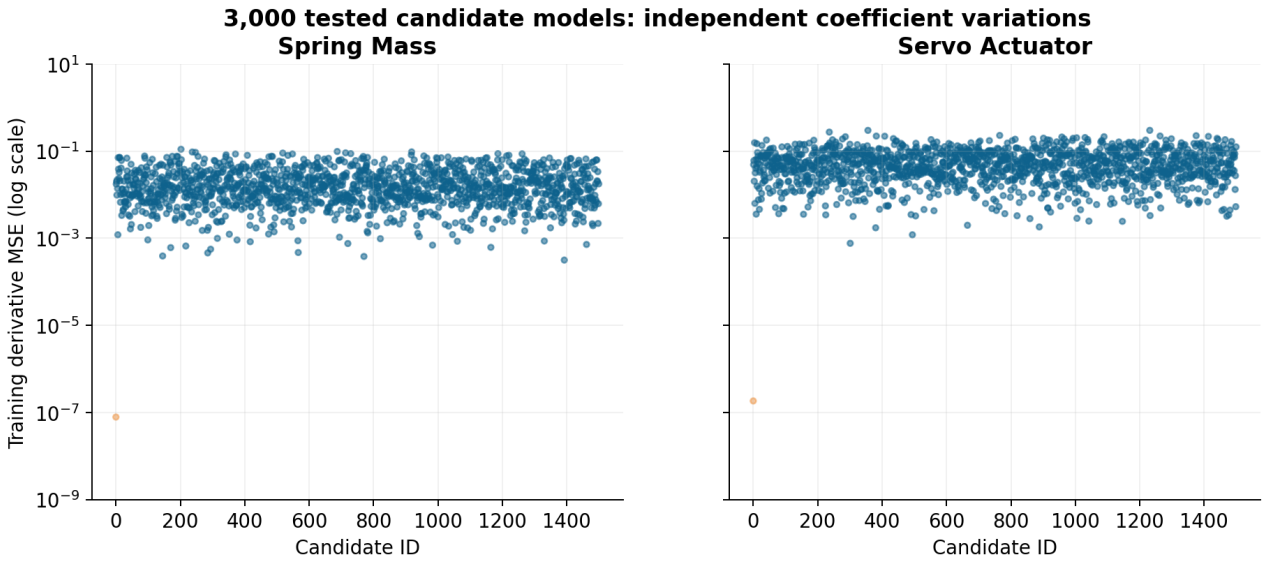


Figure 2. Training-target errors across the full candidate pool. The planted candidate appears at candidate ID 0, shown separately from perturbed alternatives.

A fixed-size shortlist of 32 candidates per system is established by training derivative error; only the shortlist is evaluated on independent validation records. No test trajectory is considered before selection.

4. Differentiation, selection and independent rollout

Training and validation targets are estimated using second-order central finite differences of the recorded velocity, with second-order one-sided estimates at the boundaries. The model search therefore operates on derivative estimates, not direct access to the exact equation values. The simulation data remain noise-free, so this test does not establish performance under realistic sensor noise.

Once a candidate is selected by validation derivative MSE, its differential equation is integrated with the C++ RK4 algorithm. The selected acceleration is evaluated by paceval at each RK4 stage. A distinct native reference uses four substeps per recorded interval, which produces a measurable nonzero numerical residual without changing the tested differential equation.

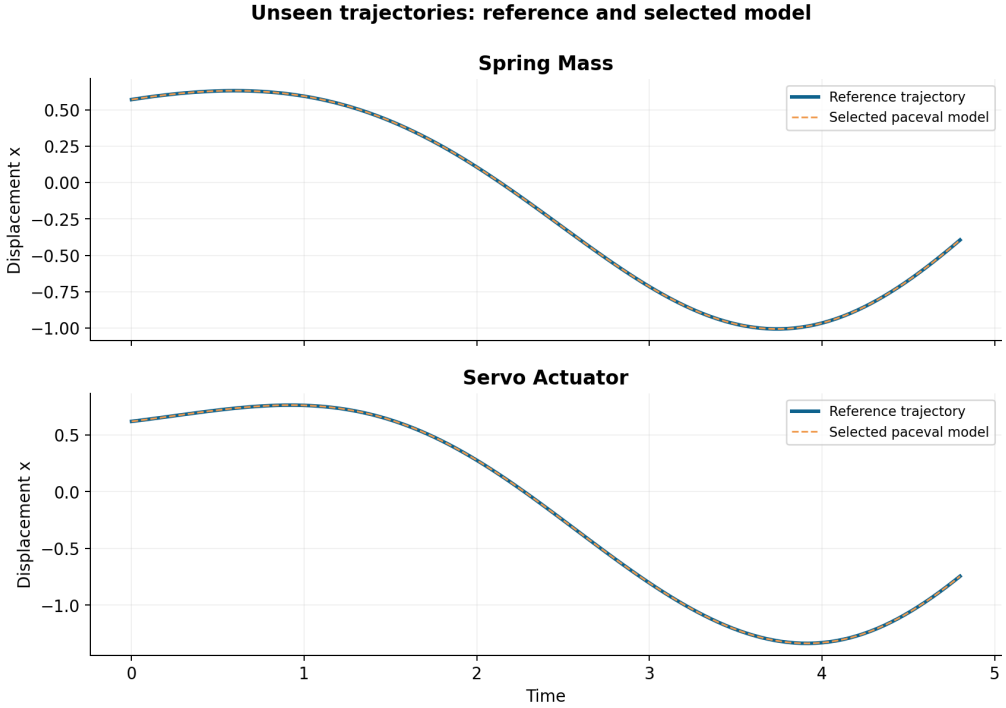


Figure 3. Selected models versus simulated reference displacement on a previously unseen excitation and initial condition.

5. Held-out dynamical behavior

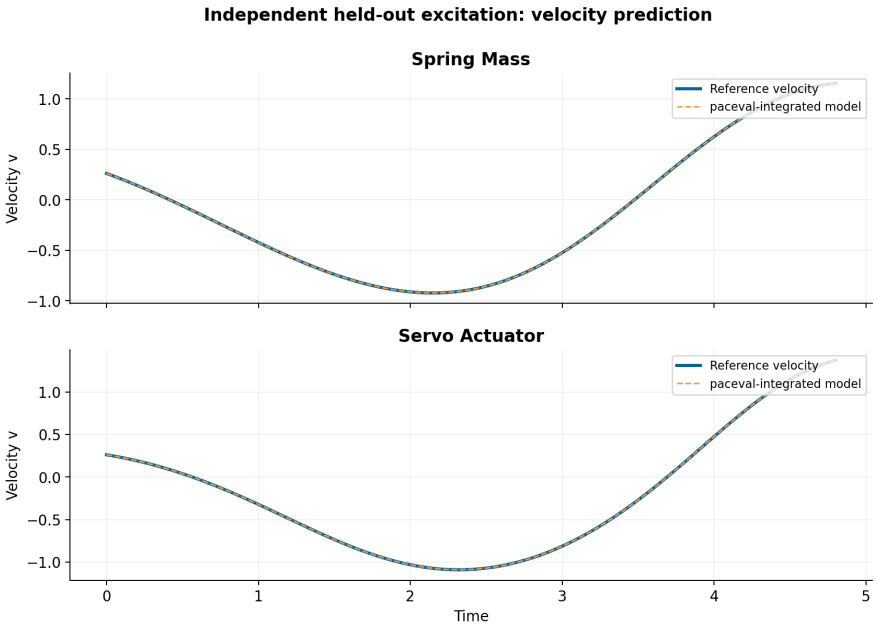


Figure 4. Velocity behavior for the second set of held-out trajectories, using the selected mathematical model rather than the original reference evaluator.

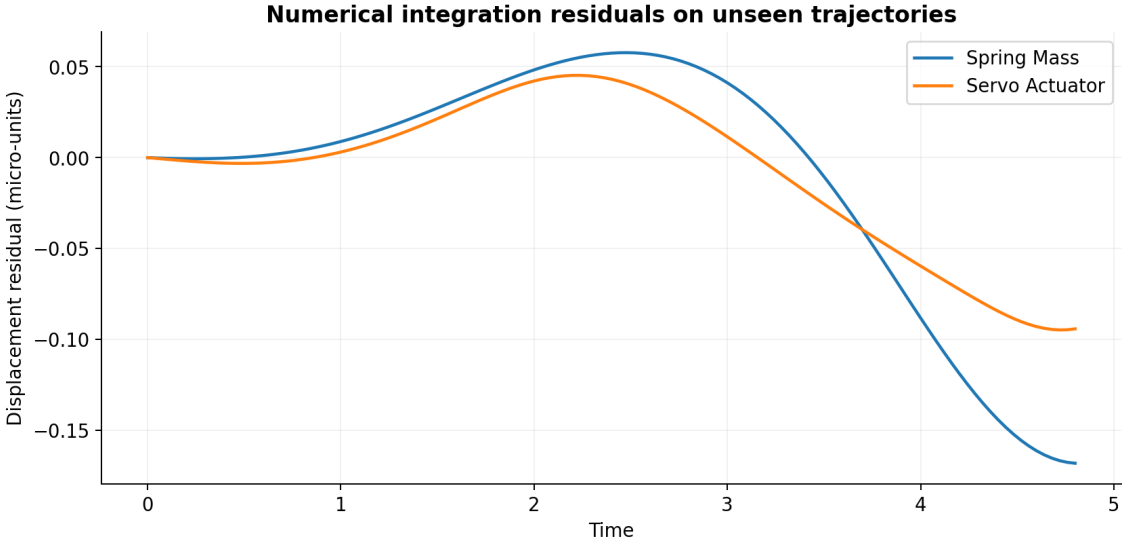


Figure 5. Nonzero displacement differences between the coarse paceval-driven RK4 simulation and the finer native reference integration. The residual scale is shown in micro-units.

6. Reproduced model-selection outcomes

System	Selected structure	Held-out x RMSE	Held-out v RMSE
Spring–mass	Cubic stiffness (family 1)	7.52×10^{-8}	8.32×10^{-8}
Servo actuator	Cubic velocity damping (family 4)	4.36×10^{-8}	3.67×10^{-8}

The resulting trajectory errors are in synthetic state units and are conditional on the planted reference candidate, the finite-difference target method, the chosen input trajectories and the numerical integration scheme. They must not be interpreted as general accuracy guarantees for other physical systems.

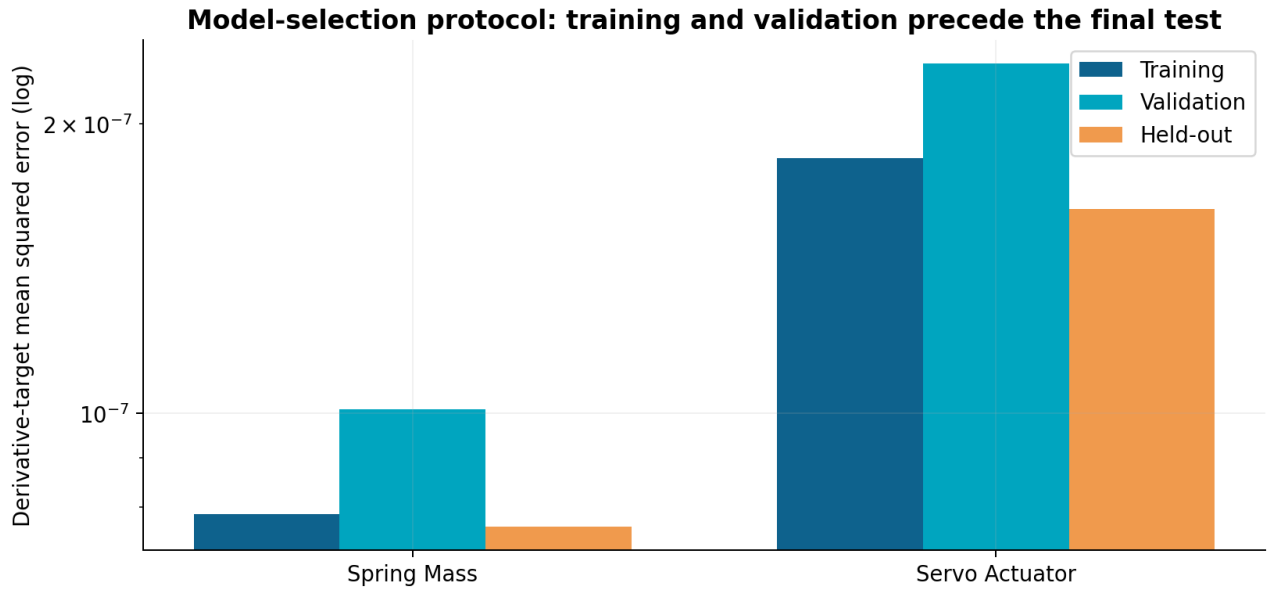


Figure 6. Training, validation and held-out derivative-target errors for the two selected mathematical families.

The held-out errors are computed only after model selection. The near-reference numerical behavior therefore supports the feasibility of repeated mathematical candidate evaluation in a controlled offline workflow.

7. Versioned mathematical model updates

A new model version is created by adjusting coefficient d while reusing the exact same paceval computation graph, because the formula structure and variable signature do not change. This is the central architectural advantage being demonstrated: a prepared mathematical function can serve many candidate parameters and later revised model versions.

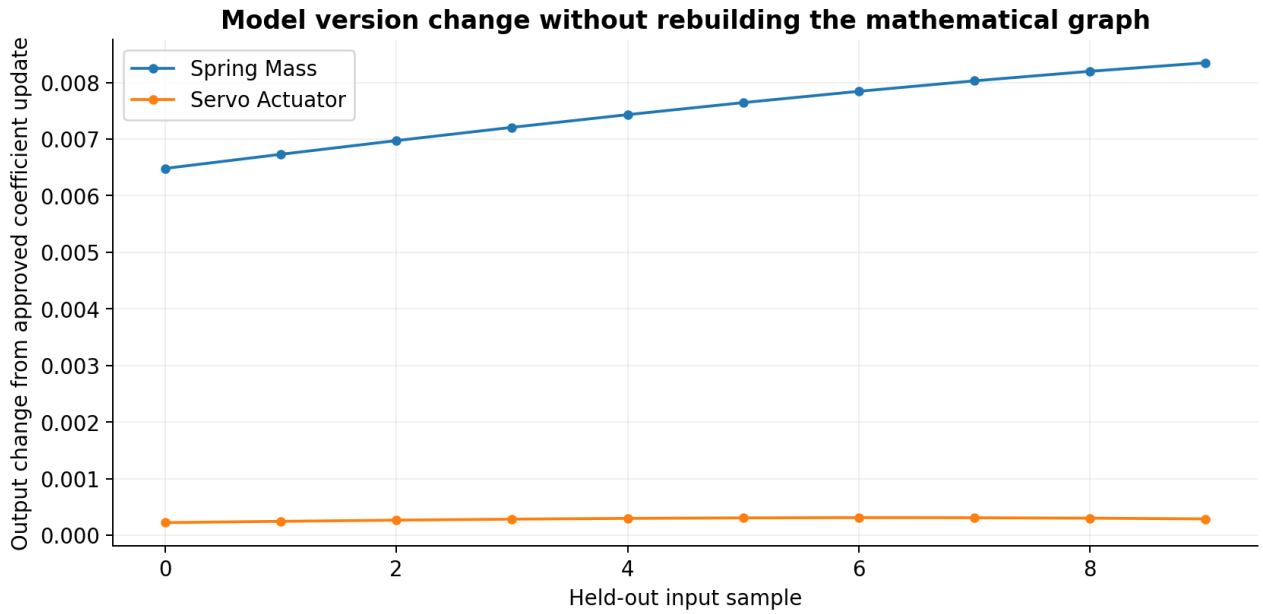


Figure 7. Effect of an explicitly recorded coefficient update on successive held-out input samples. The graph remains unchanged; the coefficient bindings differ.

8. Actual paceval API use

The C++ program compiles src/system_identification.cpp and the official src/paceval_main.cpp together, using src/paceval_main.h for API declarations. It calls paceval_InitializeLibrary and paceval_FreeLibrary for lifecycle management, paceval_CreateComputation and paceval_DeleteComputation for graph objects, paceval_dGetComputationResultExt for batch processing, and paceval_dGetComputationResult during trajectory integration. Graph and per-item error responses are checked.

The runtime libpaceval_linux_sharedLIB.so is a separately supplied dependency; it is not packaged in this public research archive.

9. Independent numerical audit

The native C++ evaluator reproduces all seven mathematical families without calling paceval. During candidate screening, 7,325 pointwise values were compared against that evaluator without tolerance violations. An independent mpmath implementation at 110 decimal digits checked 144 saved evaluations. The largest absolute high-precision difference was 2.79×10^{-16}, with all 144 values inside the declared tolerance $|\text{difference}| \leq 2 \times 10^{-9} \times (1 + |\text{reference}|)$.

An additional 24 pointwise TINC outputs were examined against the same high-precision evaluator, and 24 contained the independently calculated result. Such finite sample containment establishes the observed behavior for these inputs only. It does not certify global outward-rounded enclosure for every expression or hardware environment.

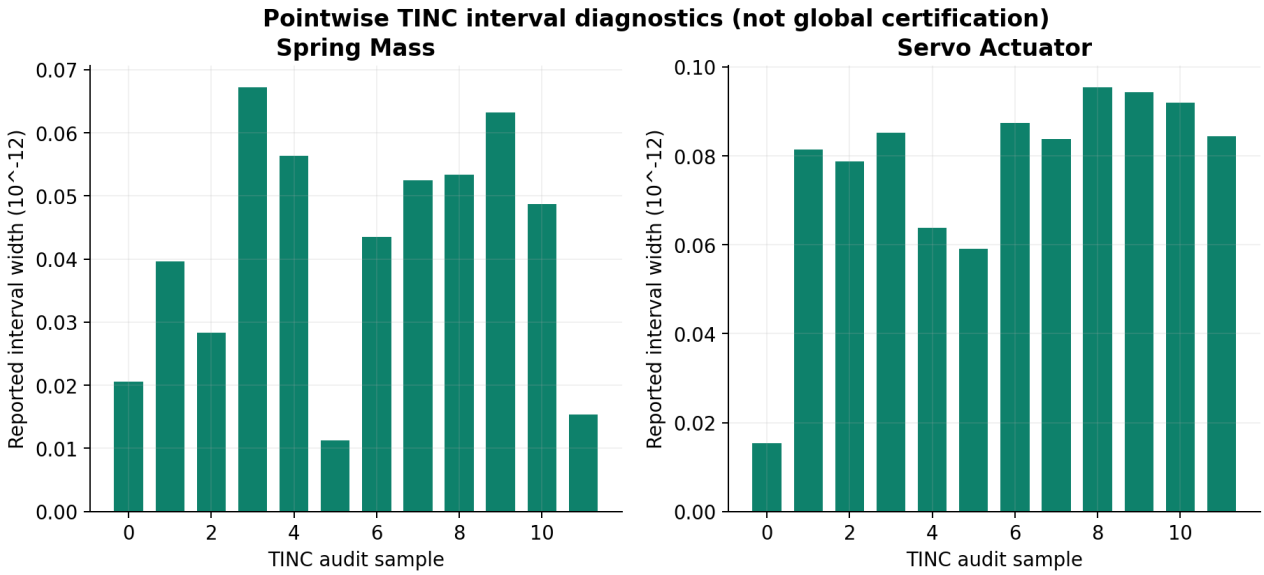


Figure 8. The measured widths of the 24 TINC pointwise intervals. These are interval diagnostics, not a claim about all possible input ranges.

10. Reproducibility controls

The random seed is 20261111. All measured values, source models, split assignments, selected candidates, model updates, numerical audit samples and machine-readable conclusions are included under examples/. Full default runs with batch sizes 37 and 128 were confirmed to produce byte-identical CSV and JSON outputs. GCC 14.2, Clang 17, AddressSanitizer and UndefinedBehaviorSanitizer builds each passed three CTest checks.

11. Source material and build instructions

Use CMake directly from the src/ directory and supply the original, compatible paceval Linux shared library externally. The same exact SDK headers and wrapper used for these tests are included. The build does not download or bundle a runtime.

```
export PACEVAL_LIB="/absolute/path/libpaceval_linux_sharedLIB.so"

cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME="$PACEVAL_LIB"

cmake --build build -j && ctest --test-dir build --output-on-failure

./build/system_identification --candidates 1500 --batch 128 --output results

python src/high_precision_reference.py results --output results/high_precision_audit.json
```

The source script src/make_figures.py reproduces the scientific PNG and SVG images. The published examples include observations.csv, candidates.csv, selection.csv, heldout_trajectories.csv, model_updates.csv, audit_samples.csv, tinc_samples.csv, validation.json and high_precision_audit.json. Every referenced path is present in this archive.

12. Scientific interpretation and next experiments

The result establishes a reusable mathematical evaluator embedded within a controlled candidate-screening and ODE-validation workflow. The next useful work includes noisy measurements with robust differentiation, unconstrained parameter identification, automated SINDy/PySINDy or symbolic-regression integration, adaptive solvers and genuine laboratory data with verified units. None of those extensions is represented as tested here.

References

Brunton, Proctor & Kutz (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. PNAS 113(15), 3932–3937. https://doi.org/10.1073/pnas.1517384113

PySINDy documentation. Practical tips: Numerical differentiation and data quality. https://pysindy.readthedocs.io/en/latest/tips.html

paceval official C/C++ API: https://paceval.com/api/ ; public project: https://github.com/paceval/paceval

Research status: working synthetic offline demonstrator; not an experimental physical discovery, a model-identifiability proof, or a comparative speed benchmark.